

Proposed Browser Vision

We propose a **feature-rich desktop browser** (Windows, Mac, Linux) that appeals to both power users and novices. It will combine the best ideas from existing browsers, with modern architecture and cutting-edge features. Key goals include: multi-process stability (like Chrome) ¹, strong security sandboxing ² ³, integrated ad/tracker blocking and privacy by default ⁴ ⁵, and a highly customizable, user-friendly UI. We will support all major web ecosystems (Chrome/Firefox extensions, Google/Facebook logins, etc.) and specifically optimize **media streaming** (4K Netflix, YouTube HD, anime sites, etc.) by adding hardware decoding, adaptive bit-rate, and full DRM support ⁶ ⁷. The browser will include built-in password management, allow biometric unlock (Windows Hello/TouchID) ⁸, and keep data as private as possible.

Architecture and Stack

- **Rendering Engine:** We will base the browser on a modern open-source engine (e.g. Chromium's Blink/V8 or Mozilla's Gecko/SpiderMonkey). Using Chromium (as Brave/Edge do) offers native Chrome-extension compatibility and Widevine DRM, but we will also ensure Firefox WebExtension support where possible. The multi-process architecture will isolate tabs, plugins, GPU, and utility tasks into separate OS processes ¹. This "one tab (or site) = one process" model means if a page crashes, it won't take down the whole browser ¹. Each process is sandboxed (limited privileges) to prevent malware from escaping a tab ² ³. Modern browsers like Chrome and Edge use this strategy, which also limits cross-site data leaks (site isolation) ³. We will implement a service-oriented design: on powerful machines, browser services (network, storage, UI) run separately for stability, while on low-memory systems we consolidate processes to save RAM (similar to Chrome's "servicification") ⁹ ².
- **Programming Languages & Frameworks:** The core will be in C++ (for performance) with critical modules (like adblock) in memory-safe Rust where possible (as Brave did to cut memory) ¹⁰. The UI can use a cross-platform toolkit (e.g. Qt or a custom Skia-based UI) to ensure native look-and-feel on each OS. We'll avoid heavyweight frameworks like Electron to keep resource usage low. For UI customization, we'll expose theming and layout via CSS/JS (like Firefox's `userChrome.css`) and support dynamic UI changes.

Key Features and Design

Performance and Efficiency

- **Resource Management:** Each tab will be a separate process to maximize responsiveness ¹. We will include process consolidation heuristics so that on memory-constrained systems, multiple same-site tabs share a process (avoiding runaway process counts) ⁹. Our dev team will aggressively optimize memory. For example, we will use techniques like Brave's new Rust adblock engine (replacing heap structures with flat binary tables) to reduce memory use ¹⁰. Integrated features (adblock, tracking protection) will be native and tightly optimized, avoiding costly extension APIs ¹¹.

We'll enable hardware acceleration for rendering and video decoding (GPU-based H.264/H.265/VP9/AV1) to offload work from CPU and improve battery life.

- **Video Streaming:** To deliver *the highest video quality*, we will support all standard streaming technologies (HTML5, HLS, DASH) and codecs. We'll integrate DRM modules (Widevine, PlayReady, etc.) for full HD/4K streaming on Netflix, Amazon Prime, Disney+, and others ⁷ ⁶. (For example, Netflix's site limits Chrome on Windows to 1080p unless hardware DRM is enabled ⁶; by bundling Widevine L1 and meeting DRM requirements, our browser can potentially unlock 4K wherever content providers allow.) We'll also include picture-in-picture, casting (Chromecast/DLNA), and an intelligent streaming pipeline: adaptive bitrate (auto-select quality based on bandwidth), prefetching of video segments, and multi-CDN fallback for smoother playback. These will be tuned for both popular sites and niche ones (e.g. anime sites with custom players) by ensuring compatibility with all major video tags and protocols. Hardware-accelerated decoding (especially for new codecs like AV1) will minimize CPU use and power consumption.
- **Multimedia and UI Features:** The browser will provide a rich UI for media: e.g. thumbnail previews in the timeline, custom controls, and easy screenshot/PIP. Media keys (play/pause on keyboard) will be supported, and we'll allow "pop-out video" even from pages lacking it. If feasible, we may support seamless handoff to external players (like opening a Twitch stream in a native app) via protocols. In short, streaming will feel as native and high-quality as possible, addressing known limits (e.g. Chrome's 1080p cap) by using the same tech (DRM and codecs) that sites expect.

Security and Privacy

- **Sandboxing & Isolation:** We will follow best-practice isolation. Each renderer (website) runs in a sandboxed OS process ². Site Isolation (separate process per site) will be default, thwarting cross-site attacks like Spectre ³. The browser itself (UI process) will run at higher privilege, but it will strictly vet communication (IPC) with renderers to block malicious actions.
- **Safe Browsing/Phishing Protection:** Like Chrome's Safe Browsing, our browser will check pages and downloads against updated threat lists ¹². If a user tries to visit a known phishing or malware site, we'll show a full-page warning. (Google's Safe Browsing protects billions of devices ¹²; we will use a similar API or partner for threat feeds.) We will also scan downloaded files (optionally) for viruses and unwanted software signatures.
- **Password Manager & Biometric Unlock:** We will build in a secure password manager (like Chrome's or Firefox's). Users' logins and secrets will be stored encrypted (using OS facilities: e.g. Windows DPAPI, macOS Keychain). Crucially, on Windows we will allow unlocking these (and WebAuthn passkeys) via Windows Hello (PIN, fingerprint, face) ⁸. For example, creating or using a passkey may prompt a Windows Hello scan, ensuring only the owner can access credentials ⁸. (Mac users will get Touch ID support for the keychain.) Syncing with a cloud (Google account or a private server) will be optional; local encryption and biometric access will ensure privacy if users choose it.
- **Default Privacy Protections:** By default, the browser will block third-party cookies, trackers, and fingerprinting scripts (similar to Brave or Firefox's strict mode) ⁵. We will include built-in, regularly updated ad & tracker blocker (see Ad Blocking below). Users can opt into or out of privacy modes:

for example, an “ultra-private” mode could disable all 3rd-party scripts and cookies. We will honor or pre-process Do Not Track, but go further by actively blocking fingerprinting vectors.

- **Additional Security:** HTTPS-only mode (warn on HTTP), strict certificate validation with HSTS support, and DNS-over-HTTPS. Code will be sandboxed, and the browser auto-updates patches. If possible, we'll integrate a VPN or Tor option for advanced users. We'll include a “Site Isolation” mode and a “strict sandbox” mode for extra security (at cost of performance). Crucially, no data will be sold or misused – unlike some competitors, we will be clear that user data stays private by design (no telemetry without consent).

Ad Blocking

- **Native Adblock & Tracker Blocking:** We'll include a first-party adblocking engine (like Brave's built-in Shields) ¹¹. Unlike extension-based blockers, a native engine can use efficient data structures and share filters across tabs. For example, Brave rewrote its adblock in Rust and cut memory use by ~75% ¹⁰. We will similarly optimize for speed and low footprint. Users can toggle adblocking on/off per site and load custom filter lists (EasyList, etc.).
- **Phishing/Malware Ad Detection:** The adblocker will also filter malicious ads and unwanted pop-ups. We'll maintain blocklists for known malware domains and crypto-miners. Since it's built-in, we can even block ads before they load, saving bandwidth.
- **Custom Content Blocking:** Users can right-click any page element to block it (“element picker”). This lets power users tailor their experience (removing cookie banners, comment sections, etc.). We'll consider automating cookie-notice hiding (similar to Brave's cookie crumbler) to improve usability.

Usability and Customization

- **User Interface:** The UI will be modern and clean for new users, but deeply configurable for experts. By default, it might resemble Chrome/Edge's minimalist look, but veterans can enable advanced toolbars, side panels, and tiling of tabs (like Vivaldi). We'll allow vertical tabs, tab grouping/pinning, and keyboard shortcuts for all actions. A “New Tab” dashboard can show customizable widgets (weather, bookmarks, news).
- **Profiles and Sync:** Multiple user profiles are supported (for families or work vs personal). Each profile syncs bookmarks, history, passwords, extensions via a cloud service (e.g. Google Account or our own). We'll provide import wizards to bring data from Chrome, Firefox, Safari, etc., making it easy to switch. We'll also support WebHistory and syncing across devices (desktop only initially, but maybe mobile later).
- **Extensions Ecosystem:** Our browser will be compatible with Chrome/Chromium extensions out of the box. We will also aim for compatibility with Firefox WebExtensions (since both use similar APIs). Users can install from the Chrome Web Store or an “Add-ons” gallery we provide. We may whitelist or verify extensions for security.
- **Developer Tools and Power Features:** For power users (devs, researchers), we'll include a robust DevTools panel (like Chrome's). Advanced features: built-in proxy settings, user agent switching,

scripting console, network throttling, etc. We'll support custom `userChrome.css` styling to restyle the UI. We could offer mouse gestures, voice search, and advanced search operators in the address bar.

- **Accessibility:** The browser will meet accessibility standards: screen reader support, high-contrast themes, adjustable font zoom, and easy keyboard navigation. Users can resize the UI or use built-in reading mode (simplified text view) for better legibility.

Video Streaming Optimization

To deliver **best-in-class streaming**, the browser will implement:

- **High-Quality Playback:** We will include all major video codecs (H.264, H.265/HEVC, VP9, AV1, etc.) and ensure hardware acceleration is used. Using GPU decoders wherever possible dramatically reduces CPU load and improves battery life ¹³. For premium content (Netflix, Prime, Disney+), we will license and embed DRM modules (Widevine, PlayReady). Netflix, for example, uses Widevine DRM to control quality; on Windows, only Edge + PlayReady enables 4K ⁶. By bundling these DRMs, our browser can achieve the highest allowed quality on each platform (e.g. 4K on Netflix via Widevine L1 ⁷ ⁶, 4K Disney+ if possible).
- **Adaptive Streaming:** We'll fully support DASH and HLS with multi-track ABR. The browser will automatically select the best resolution given bandwidth and CPU. We may implement prefetching of future segments or use multiple connections to speed up loading. Users can set a maximum quality or a data-saving mode.
- **Network Efficiency:** The browser will use HTTP/3 (QUIC) and optimized TCP stacks to improve video throughput. We'll consider parallel connection techniques (like fetching from multiple CDN endpoints or sources) to avoid stalls. A built-in speed-test could help tune performance.
- **Media Integration:** For video-heavy sites, the browser will provide special UI (e.g. not playing in a tiny iframe). We'll detect full-page players and unlock native controls. In full-screen or PiP mode, controls will be easily accessible. We'll support hardware HDR or surround sound output if available.
- **Compatibility:** All mainstream streaming sites and many niche ones (anime streaming, sports streaming, etc.) will be tested. We may include user-agent or feature-switches for tricky sites. If a site uses a non-HTML5 player, we'll work to bridge it or recommend alternatives.

By focusing on these streaming optimizations, we aim to **outperform other browsers** in media playback quality and performance.

Security, Privacy and Ad Blocking (continued)

- **Ad/Tracker Blocking Integration:** By building adblock into the browser itself, we can block ads before rendering and share filter data across tabs. This native approach (as used by Brave ¹¹) avoids the overhead of extension APIs. Users get privacy and faster page loads out-of-the-box.

- **Privacy Dashboard:** We will include a “Privacy Report” that shows how many ads/trackers were blocked and what data was prevented from leaking. Users can configure per-site exceptions.
- **Windows Hello and Passkeys:** We will leverage modern auth standards. For example, when a site offers WebAuthn passkeys, the browser can use Windows Hello to create and unlock them ⁸. This gives a seamless yet secure login experience: only the authorized user’s face/fingerprint can unlock the saved credentials. On macOS, Touch ID and iCloud Keychain integration will provide similar passkey support.

Development Plan

1. **Requirements & Design:** Survey current browsers (Chrome, Firefox, Brave, Vivaldi, Edge) and user feedback to compile feature list. Create UI/UX prototypes, define architecture (multi-process design, security model).
2. **Core Implementation:** Choose base engine (likely Chromium). Set up the multi-process framework and basic UI shell. Implement tab management, address bar (omnibox), profile system, bookmarks/history storage.
3. **Security Layer:** Integrate sandboxing and site-isolation as per design. Plug in Safe Browsing APIs to check URLs and downloads ¹². Add certificate pinning and auto-updates.
4. **Media Engine:** Incorporate media playback stack (codecs, decoders). License DRM modules (Widevine/PlayReady). Implement adaptive streaming support (HTML5 video with DASH/HLS). Test on Netflix, YouTube, Amazon, Crunchyroll, HiAnime, etc., ensuring highest quality. Optimize video pipeline.
5. **Ad/Privacy Tools:** Develop the built-in ad/tracker blocker using a high-performance engine (Rust if possible). Bundle filter lists. Create UI for Shields toggling. Implement privacy protections (cookie controls, anti-fingerprinting).
6. **User Features:** Build password manager and sync, integrating Windows Hello/Touch ID. Develop extensions support (Chrome store, etc.). Add UI customization (themes, custom toolbar).
7. **Testing:** Perform extensive QA on all platforms. Security audits and fuzzing to find vulnerabilities. Performance profiling to tune memory/CPU. Beta releases to community for feedback.
8. **Launch & Iterate:** Release the browser, likely as a downloadable with auto-updates. Continue to refine features, push updates for new standards (e.g. emerging codecs, new web APIs), and respond to user needs.

Throughout development, we will emphasize **security by design and user choice**. Users should feel safe (thanks to sandboxing and Safe Browsing) and empowered (via customization and privacy defaults). As a desktop-only browser, we’ll leverage the full power of each OS (multi-threading, GPU, native biometrics) while carefully managing resources, following examples like Brave’s memory-optimized adblock ¹⁰. In the end, this browser aims to unify **the best of all worlds**: top-tier streaming quality, rock-solid security, and maximal customizability, giving veterans and newbies alike the features they’ve been waiting for.

Sources: Our design draws on modern browser research and implementation details: Chrome’s multi-process/sandbox model ¹ ², Google Safe Browsing stats ¹², Brave’s built-in adblock optimizations ¹⁰ ¹¹, privacy-blocking features ⁴ ⁵, Windows Hello/passkey support ⁸, and Netflix’s quality constraints (1080p vs 4K) ⁶ ⁷. These inform our approach to build “the ultimate” desktop browser.

1 9 Inside look at modern web browser (part 1) | Blog | Chrome for Developers

<https://developer.chrome.com/blog/inside-browser-part1>

2 3 4 5 Browser Security Tools: Key Features and Top 13 Options in 2026

<https://www.venn.com/learn/browser-security/browser-security-tools/>

6 Netflix supported browsers and system requirements | Netflix Help Center

<https://help.netflix.com/en/node/30081>

7 Widevine - Wikipedia

<https://en.wikipedia.org/wiki/Widevine>

8 Windows 11 expands passkey manager support - Windows IT Pro Blog

<https://techcommunity.microsoft.com/blog/windows-itpro-blog/windows-11-expands-passkey-manager-support/4467572>

10 11 Brave overhauls adblock engine, cutting its memory consumption by 75% | Brave

<https://brave.com/privacy-updates/36-adblock-memory-reduction/>

12 Safe Browsing – Google Safe Browsing

<https://safebrowsing.google.com/>

13 Why can't I watch Netflix in Ultra HD on my Chrome browser? | by Daniel Kim | DoveRunner Tech Blog | Medium

<https://medium.com/pallycon/why-cant-i-watch-netflix-in-ultra-hd-on-my-chrome-browser-525933dad5bb>